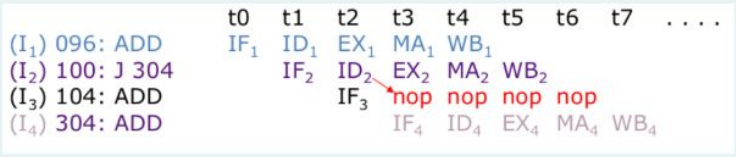
Se consideră execuția programului de mai jos în bandă de asamblare. Se presupune că speculăm valoarea PC+4. Instrucțiunea 2 nu face STALL deoarece avem speculație dar de ce instrucțiunea 3 introduce doar fetch ?

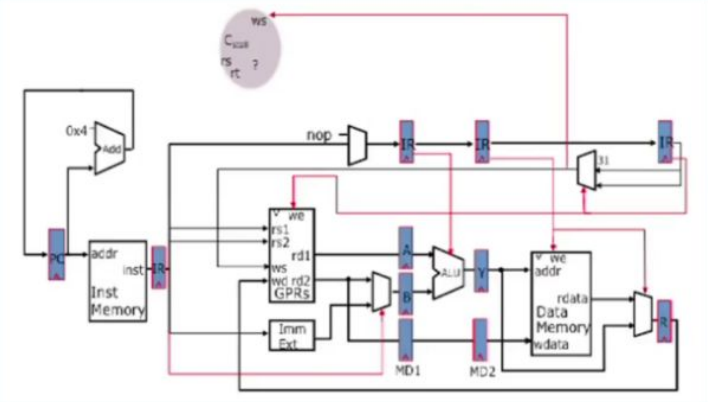


- a. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Speculația fiind greșită, instrucțiunea 3 trebuie eliminată din banda de asamblare.
- b. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard structural, ca atare va trebui eliminată din banda de asamblare
- c. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard de control, ca atare va trebui eliminată din banda de asamblare
- d. Programul nu poate fi executat în bandă de asamblare deoarece prezintă instrucțiuni ce nu pot fi executate în bandă de asamblare
- e. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard de date, ca atare va trebui eliminată din banda de asamblare

Când poate apărea o anti-dependență ?

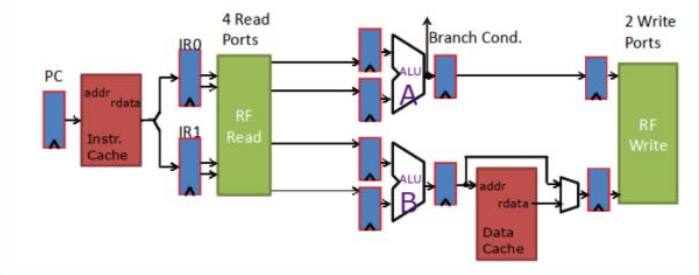
- [ ] a. O anti-dependență nu este posibil să apară
- [ ] b. O anti-dependență poate apărea atunci când instrucțiunile nu sunt executate în ordine
- [ ] c. O anti-dependență poate apărea atunci când mai multe instrucțiuni sunt executate în același moment de timp
- [ ] d. O anti-dependență poate apărea atunci avem un hazard structural
- [ ] e. O anti-dependență poate apărea doar în faza de IF

Se consideră următoarea implementare a unui procesor pentru determinarea semnalului C_STALL. Ce trebuie verificat în acest caz ?



- [ ] a. Se verifică identificatorul registrului
- [ ] b. Practic se verifică câmpul RS din cadrul unei instrucțiuni MIPS
- [ ] c. Se verifică valoarea registrului
- [ ] d. Practic se verifică câmpul RD din cadrul unei instrucțiuni MIPS
- [ ] e. Practic se verifică câmpul **immediate16** din cadrul unei instrucțiuni MIPS
- [ ] f. Practic se verifică câmpul OPCODE din cadrul unei instrucțiuni MIPS
- [ ] g. Practic se verifică câmpul **immediate26** din cadrul unei instrucțiuni MIPS
- [ ] h. Practic se verifică câmpul RT din cadrul unei instrucțiuni MIPS

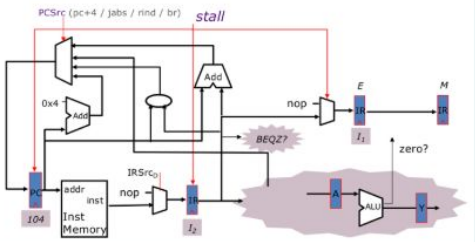
În cazul unui procesor superscalar IO cu 2 căi ca în figura de mai jos sunt planificate spre execuție 2 instrucțiuni. O instrucțiune LW în registrul IR0 și o instrucțiune ADD în registrul IR1. Ce mai trebuie adăugat ca să poată fi posibilă execuția celor 2 instrucțiuni ?



- [ ] a. O logică suplimentară care să permită duplicarea operandului/operanzilor uneia sau mai multor instrucțiuni
- [ ] b. Folosirea a două decodificatoare
- [ ] c. O logică suplimentară care să permită switch între instrucțiuni
- [ ] d. Duplicarea logicii de control

Execuția următorului program în bandă de asamblare presupune eliminarea instrucțiunilor 104 și 108. Pentru aceasta este nevoie:

I1 – 096 ADD
I2 – 100 ***BEQZ*** r1 + 200
I3 – 104 ADD
108 .....
I4 – 304 ADD



- [ ] a. Introducerea unui MUX suplimentar
- [ ] b. Introducerea de noi registre
- [ ] c. Introducerea de NOP-uri
- [ ] d. Introducerea unei memorii de instrucțiuni
- [ ] e. Schimbată condiția pentru IRSrc
- [ ] f. Introducerea unei memorii de date

Se consideră următoarea execuție a unui program în bandă de asamblare. De ce este nevoie de introducerea NOP-urilor în execuția programului cu toate că programul nu prezintă hazard-uri?

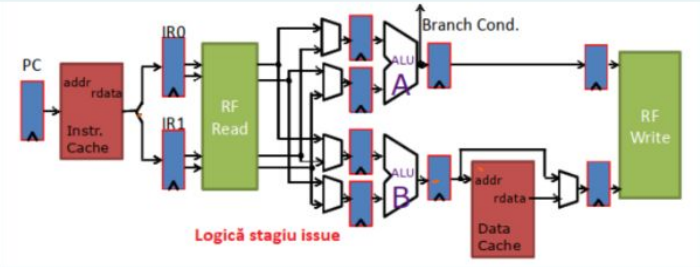


- a. Se evită hazardul de control deoarece trebuie determinată instrucțiunea 2
- b. Programul nu prezintă hazarduri și ca atare nu trebuiesc puse NOP-uri
- c. Se evită hazardul de date deoarece trebuie determinată instrucțiunea 2
- d. Nu trebuie puse NOP-uri
- e. Se evită hazardul structural deoarece trebuie determinată instrucțiunea 2

O dependență ieșire când poate apărea ?

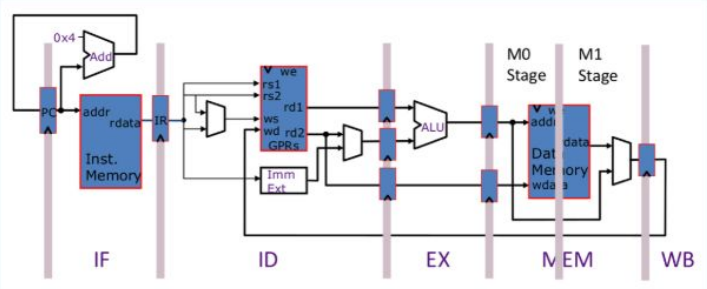
- [ ] a. O dependență de ieșire poate apărea doar în faza de IF
- [ ] b. O dependență de ieșire poate apărea doar în faza de ID
- [ ] c. O dependență de ieșire nu este considerată un hazard
- [ ] d. O dependență de ieșire poate apărea în cazul execuției a 5 sau mai multe operații
- [ ] e. O dependență de ieșire poate apărea în cazul execuției a 2 operații care necesită toți ciclii de ceas

În cazul unui procesor superscalar IO cu 2 căi ce conține faza de FETCH ?



- [ ] a. În faza de FETCH vom avea PC și PC+4
- [ ] b. În faza de FETCH vom avea decodificarea instrucții
- [ ] c. În faza de FETCH vom avea determinarea instrucții
- [ ] d. În faza de FETCH vom avea două instrucțiuni simultan
- [ ] e. În faza de FETCH vom avea eliminarea instrucțiunilor JAL și JALR

Se consideră următorul procesor și execuția următorului program: ADD, LW, LW, ADD. Există hazard structural pe primele 2 instrucțiuni la M0 și M1? Dar pe ultimele două instrucțiuni la M0 și M1?

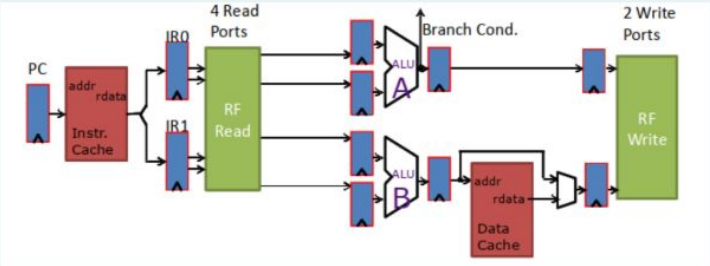


- [ ] a. Nu există hazard structural pe ultimele 2 instrucțiuni la M0 și M1
- [ ] b. Există hazard structural pe ultimele 2 instrucțiuni la M0 și M1
- [ ] c. Ar fi hazard structural dacă instrucțiunile ar fi identice
- [ ] d. Există hazard structural pe primele 2 instrucțiuni la M0 și M1
- [ ] e. Nu există hazard structural pe primele 2 instrucțiuni la M0 și M1

În cazul unei benzi de asamblare ideale:

- [ ] a. Putem partaja resurse fără restricții între stagii
- [ ] b. Ordinea stagiilor poate fi modificată
- [ ] c. Putem partaja resurse cu restricții între stagii
- [ ] d. Nu putem partaja resurse între stagii
- [ ] e. Ordinea stagiilor nu poate fi modificată

În cazul unui superscalar IO pe 2 căi registrul RF are nevoie de 4 porturi de intrare, așa cum se poate observa în figura de mai jos. De ce ?

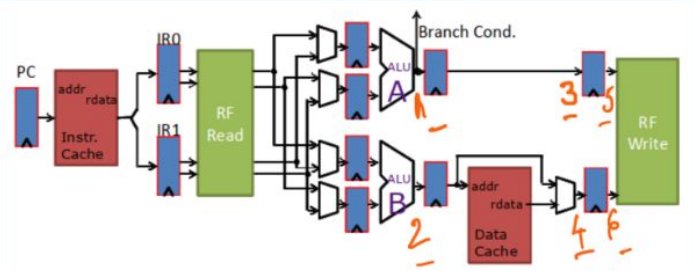


- a. Deoarece într-un astfel de procesor citirea se poate selectiv
- b. Deoarece conținutul registrelor IR nu pot fi simultan oferite la ieșire ci doar la ciclii de ceas consecutivi
- c. Deoarece un astfel de procesor poate executa doar operații cu întregi și salturi
- d. Deoarece registrul de scriere RF are 2 porturi iar 4 este putere a lui 2
- e. Deoarece numărul de instrucțiuni ce poate fi executat este dublu

ISA este un layer abstract:

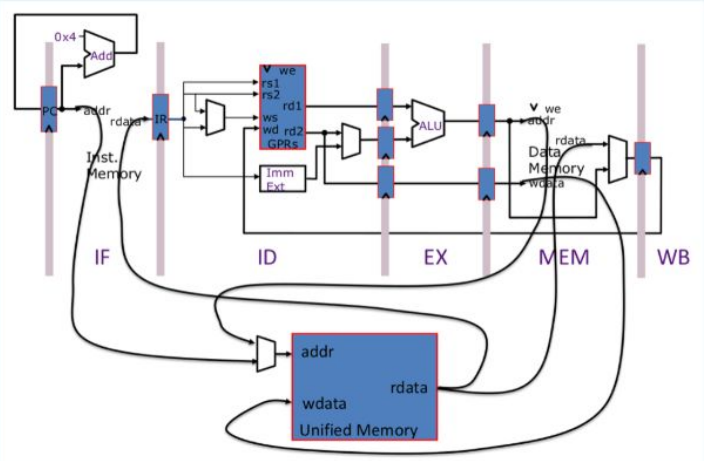
- [ ] a. menit a se schimba / modifica nu foarte des
- [ ] b. menit a se schimba / modifica foarte des
- [ ] c. noțiune neprezentată la curs
- [ ] d. oferit software-ului
- [ ] e. oferit hardware-ului

În cadrul unui procesorului de mai jos, a fost stabilit la curs că putem introduce logică pentru bypass în 6 puncte conform figurii de mai jos. Logica selectării acestor puncte crește odată cu mărirea dimensiunilor benzilor de asamblare. Cum eliminăm acest inconvenient ?



- [ ] a. Avem în vedere o execuție a instrucțiunilor într-o manieră OOO
- [ ] b. Nu aplicăm full bypass
- [ ] c. Eliminarea unor anumite resurse hardware
- [ ] d. Implementăm clustere superscalare
- [ ] e. Facem bypass doar la anumite locații

Se consideră un procesor implementat conform figurii de mai jos și execuția următoarelor instrucțiuni LW, ADD, ADD, ADD. În execuția programului trebuie introdus un stall ? Daca da, unde ?

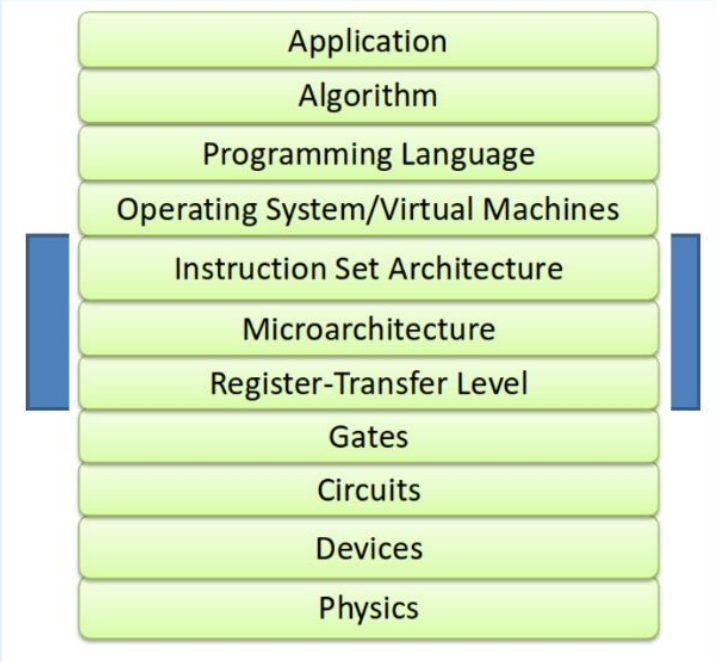


- [ ] a. Trebuie introdus stall
- [ ] b. Dacă avem stall atunci stall trebuie făcut pe a doua instrucțiune ADD
- [ ] c. Dacă avem stall atunci stall trebuie făcut pe prima instrucțiune ADD
- [ ] d. Dacă avem stall atunci stall trebuie făcut pe ultima instrucțiune ADD
- [ ] e. Nu contează introducerea de stall deoarece este un hazard structural si el nu poate fi evitat
- [ ] f. Nu trebuie introdus stall

O **întrerupere** sau o excepție este denumită **precisă** prin definiție dacă:

- [ ] a. apare doar în cadrul unui procesor OOO
- [ ] b. terminarea execuției unei instrucțiuni se face înainte de a începe execuția următoarei instrucțiuni
- [ ] c. execuția unei instrucțiuni se face în paralel cu execuția următoarei instrucțiuni
- [ ] d. starea procesorului salvat corespunde modelului secvențial de execuție a programului
- [ ] e. apariția unei întreruperi imprecise nu este posibilă

Se consideră straturile de abstractizare din figura de mai jos. Microarhitectura se referă la:



- a. modalitatea de construire a unui chip cât mai general care să acopere toate necesitățile unui utilizator
- b. organizarea sistemelor de calcul strict orientată pe consumul de putere
- c. noțiunea nu a fost prezentată la curs
- d. organizarea sistemelor de calcul
- e. modalitatea de construire a unui chip specific

este important să avem compilatoare cât mai inteligente deoarece:

- [ ] a. eliminăm mai ușor instrucțiunile care nu sunt definite în ISA
- [ ] b. eliminăm mai ușor instrucțiunile care sunt interdependente
- [ ] c. astfel eliminăm mai ușor instrucțiunile redundante
- [ ] d. eliminăm mai ușor instrucțiunile care nu ne sunt utile
- [ ] e. eliminăm mai ușor instrucțiunile mai lungi

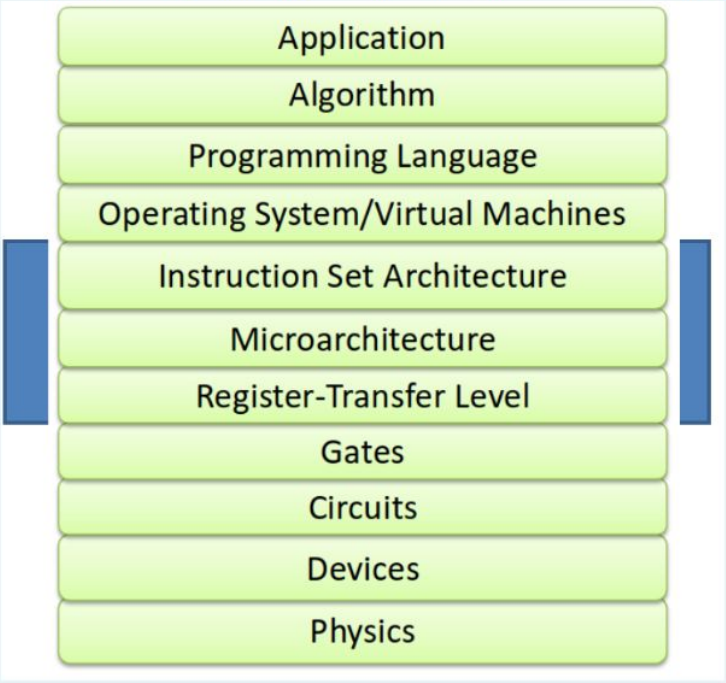
În cazul în care dorim să facem bypass între clustere superscalare vom avea nevoie de:

- a. 3 extra cicluri de ceas
- b. 2 extra cicluri de creas
- c. nu putem face această operație deoarece se va genera un hazard structural
- d. 4 extra cicluri de ceas
- e. un extra ciclu de ceas

La început fiecare procesor avea (până în 1960):

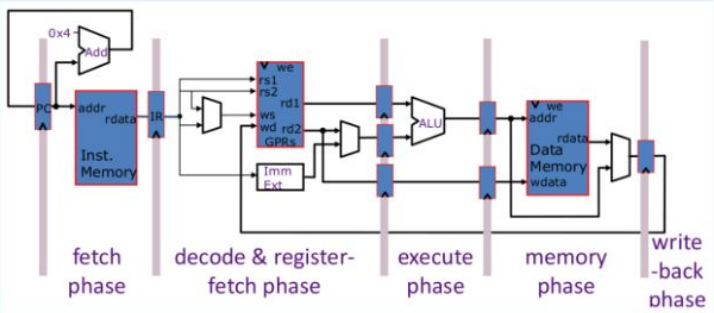
- [ ] a. propriul său set de instrucțiuni
- [ ] b. un sistem de memorie cache organizat pe nivele pentru obținerea unei performanțe mărite
- [ ] c. un sistem de memorie cache organizat pe minim două nivele pentru obținerea unei performanțe mărite
- [ ] d. propriul său sistem de I/O și storage secundar
- [ ] e. propriile sale asambloare, compilatoare și librării

Conform figurii de mai jos există un layer denumit "Register-Transfer Level". Acest nivel permite descrierea unui circuit/dispozitiv electronic prin:



- a. intermediul limbajului VHDL
- b. prin intermediul limbajului C++
- c. intermediul limbajului Verilog
- d. prin intermediul limbajului Python

Se consideră calea de date MIPS prevăzută cu bandă de asamblare. Pentru fiecare stagiu al benzii de asamblare se consideră câte un timp de execuție astfel: t_IM , t_RF , t_ALU , t_DM , t_RW. Frecvența ceasului care va controla banda de asamblare **cel mai probabil** va fi oferită de:



a. t_RW

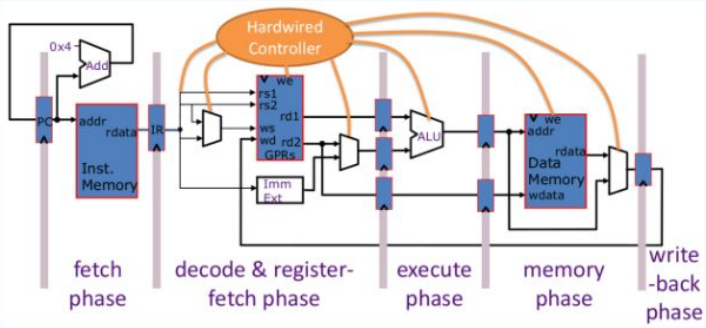
b. t_ALU

c. t_DM

d. t_IM

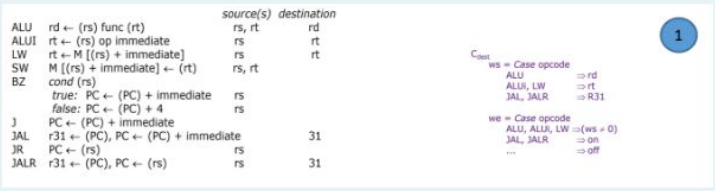
e. t_RF

În cazul căii de date a unui procesor MIPS cu bandă de asamblare una din posibilitățile de implementare ale controlului se prezintă în figura de mai jos. Această implementare conduce la următoarele concluzii:



- a. nu este o implementare bună deoarece unitatea de control nu este proiectată într-o manieră de tip bandă de asamblare
- b. nu este necesară introducerea de registre pentru a fi capabili să executăm o instrucțiune în mai mulți ciclii de ceas
- c. instrucțiunile nu pot fi executate în bandă de asamblare
- d. fără anumite modificări CPI-ul va crește - va trebui să adăugăm resurse hardware
- e. este necesară introducerea de registre pentru a fi capabili să executăm o instrucțiune în mai mulți ciclii de ceas

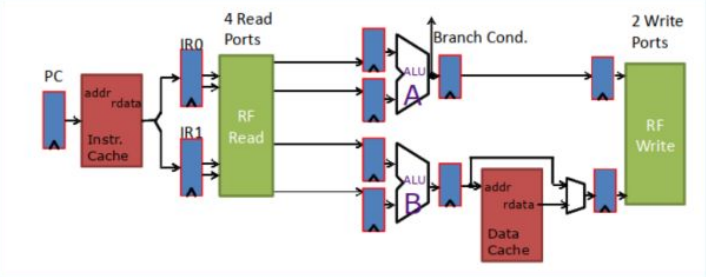
Primul pas în determinarea semnalului C_STALL este determinarea semnalului C_DEST așa cum se prezintă în figura de mai jos. De ce este nevoie de ws și de we ? Dar de CASE ?



		source(s)	*destination*
ALU	rd ← (rs) func (rt)	rs, rt	rd
ALUI	rt ← (rs) op immediate	rs	rt
LW	rt ← M [(rs) + immediate]	rs	rt
SW	M [(rs) + immediate] ← (rt)	rs, rt	
BZ	*cond* (rs)		
	true: PC ← (PC) + immediate	rs	
	false: PC ← (PC) + 4	rs	
J	PC ← (PC) + immediate		
JAL	r31 ← (PC), PC ← (PC) + immediate		31
JR	PC ← (rs)	rs	
JALR	r31 ← (PC), PC ← (rs)	rs	31

- a. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se scrie o destinație
- b. CASE este folosit pentru a identifica tipul instrucțiunii. ws este în plus, se poate și fără el. we este semnalul care ne spune dacă este nevoie să scriem sa nu.
- c. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se scrie o destinație sau nu
- d. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se citește o destinație sau nu
- e. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se citește o destinație

În cazul unui procesor superscalar IO cu 2 căi ca în figura de mai jos sunt planificate spre execuție 2 instrucțiuni. O instrucțiune LW în registrul IR0 și o instrucțiune ADD în registrul IR1. Se pot ele executa ?



- a. Nu deoarece registrul IR0 are 4 ieșiri iar operația LW are un singur operand
- b. Da, deoarece procesorul este prevăzut cu 2 căi de execuție
- c. Nu
- d. Da

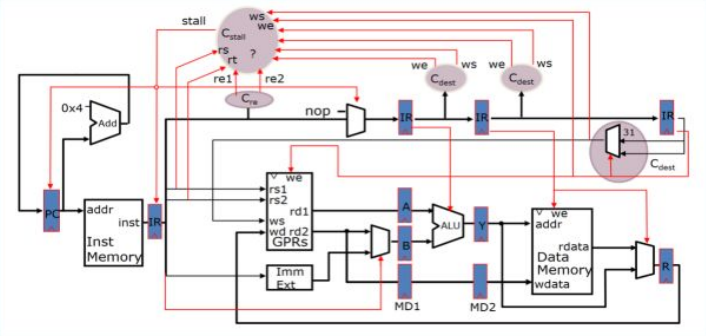
Când poate apărea o anti-dependență ?

- [ ] a. O anti-dependență nu este posibil să apară
- [ ] b. O anti-dependență poate apărea atunci când instrucțiunile nu sunt executate în ordine
- [ ] c. O anti-dependență poate apărea atunci când mai multe instrucțiuni sunt executate în același moment de timp
- [ ] d. O anti-dependență poate apărea atunci avem un hazard structural
- [ ] e. O anti-dependență poate apărea doar în faza de IF

Misiunea unui arhitect de calculatoare este aceea de a lua decizii cât mai subtile de proiectare pentru a îndeplini anumite cerințe precum:

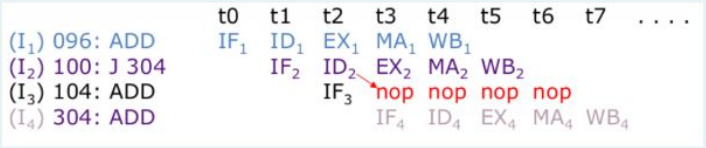
- [ ] a. consum de putere cât mai mare pentru a face calculatorul să ruleze cât mai rapid
- [ ] b. o durată de viață cât mai mare a arhitecturi
- [ ] c. consum de putere cât mai mic pentru a face calculatorul să ruleze cât mai rapid
- [ ] d. ușurință în programare
- [ ] e. o durată de viață cât mai mică a arhitecturi pentru a permite conceptelor noi să fie imediat exploatate

Se consideră implementarea unui procesor ca în figura de mai jos. Când nu trebuie trebuie verificată condiția de STALL și de ce ?



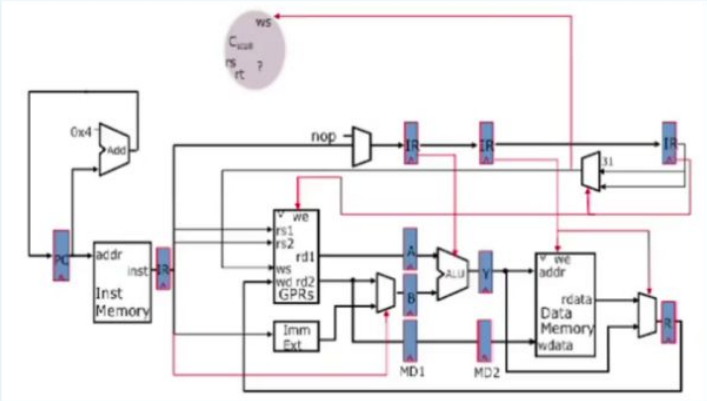
- a. Condiția de STALL nu trebuie verificată în cazul instrucțiunilor în care operandul 2 este inclus în instrucțiune
- b. Condiția STALL nu trebuie verificată în cazul instrucțiunilor care nu citesc un registru
- c. În cazul instrucțiunii SW deoarece nu scrie nici un registru
- d. Condiția de STALL trebuie verificată întotdeauna
- e. În cazul instrucțiunii LW deoarece se scrie un registru

Se consideră execuția programului de mai jos în bandă de asamblare. Se presupune că speculăm valoarea PC+4. Instrucțiunea 2 nu face STALL deoarece avem speculație dar de ce instrucțiunea 3 introduce doar fetch ?



- a. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Speculația fiind greșită, instrucțiunea 3 trebuie eliminată din banda de asamblare.
- b. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard structural, ca atare va trebui eliminată din banda de asamblare
- c. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard de control, ca atare va trebui eliminată din banda de asamblare
- d. Programul nu poate fi executat în bandă de asamblare deoarece prezintă instrucțiuni ce nu pot fi executate în bandă de asamblare
- e. Conform enunțului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard de date, ca atare va trebui eliminată din banda de asamblare

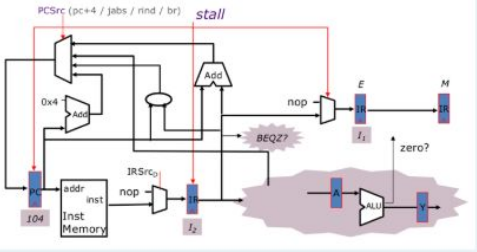
Se consideră implementarea procesorului de mai jos. În execuția unui program trebuie introdus STALL atunci când:



- a. RT al instrucțiunii aflate în stagiul EX nu este egal cu OPCODE al unei instrucțiuni necomise
- b. RT al instrucțiunii aflate în stagiul IF nu este egal cu OPCODE al unei instrucțiuni necomise
- c. RS al instrucțiunii aflate în stagiul ID nu este egal cu RD al unei instrucțiuni necomise
- d. RT al instrucțiunii aflate în stagiul EX egal cu OPCODE al unei instrucțiuni necomise
- e. **immediate16** al instrucțiunii aflate în stagiul IF nu este egal cu **immediate26** al unei instrucțiuni necomise
- f. RT al instrucțiunii aflate în stagiul IF egal cu OPCODE al unei instrucțiuni necomise
- g. RS al instrucțiunii aflate în stagiul ID egal cu RD al unei instrucțiuni necomise
- h. **immediate16** al instrucțiunii aflate în stagiul IF egal cu **immediate26** al unei instrucțiuni necomise

Pentru execuția următorului program în bandă de asamblare se presupune că instrucțiunea 3 are nevoie de STALL și în același moment de timp trebuie să facem KILL. Este important ca instrucțiunea din stagiul DECODE să fie invalidă ?

I1 – 096 ADD
I2 – 100 ***BEQZ*** r1 + 200
I3 – 104 ADD
108 .....
I4 – 304 ADD

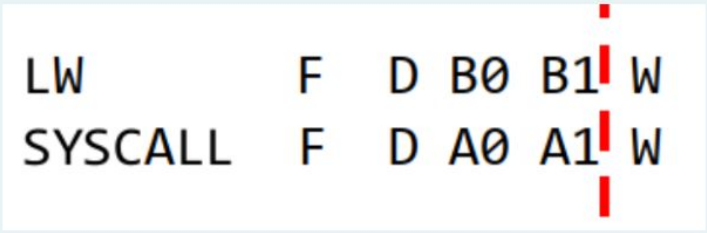


- [ ] a. Subiect netratat la curs
- [ ] b. Nu, deoarece STALL-ul trebuie să aibă prioritate minimă
- [ ] c. Da, deoarece STALL-ul trebuie să aibă prioritate maximă
- [ ] d. Nu
- [ ] e. Nu are nici o importanță invaliditatea instrucțiunii din stagiul de DECODE
- [ ] f. Da

Modul de execuție al unei întreruperi presupune:

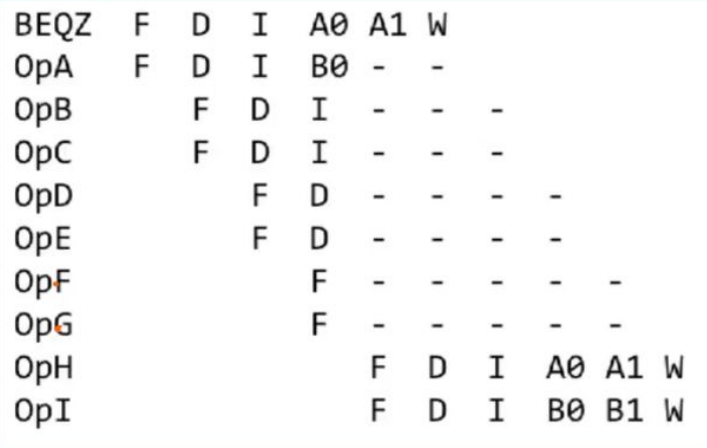
- a. Execuția unei întreruperi presupune obligatoriu existența unei benzi de asamblare
- b. Redirecționarea controlului fluxului de instrucțiuni se va face pentru o perioadă limitată de timp
- c. Redirecționa controlul fluxului de instrucțiuni într-o altă zonă
- d. La expirarea intervalului de timp necesar executării unei întreruperi se revine înapoi
- e. Execuția unei întreruperi presupune obligatoriu un procesor superscalar IO

Se consideră execuția a două instrucțiuni ca în figura de mai jos. În cazul în care instrucțiunile sunt executate altfel decât sunt prezentate în figura de mai jos ce trebuie făcut ?



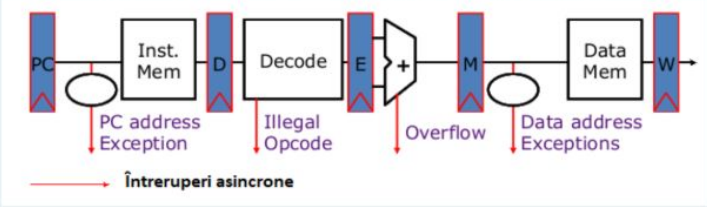
- [ ] a. eliminăm instrucțiunea LW
- [ ] b. ignorarea instrucțiunii SYSCALL
- [ ] c. ignorarea rezultatului instrucțiunii LW
- [ ] d. Nu trebuie făcut nimic
- [ ] e. eliminăm instrucțiunea SYSCALL

Se consideră execuția unui program conform figurii de mai jos. După cum se poate observa din execuția programului, instrucțiunea BEQZ este luată abia în etapa A0 ceea ce implică ca următoarele 7 instrucțiuni să fie KILL. Acest lucru conduce la un impact foarte mare asupra CPI-ului. Ce soluții avem de îmbunătățire ?



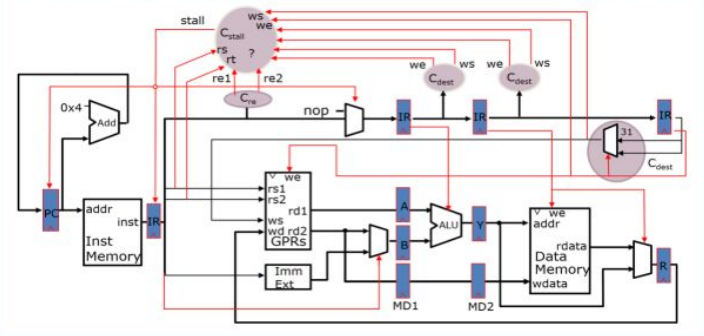
- [ ] a. Nu avem nici o soluție viabilă pentru acest tip de procesor
- [ ] b. mărirea numărului de stagii de pipe care preced stagiul A0 pentru obținerea unei predicții cât mai bune
- [ ] c. mărirea numărului de stagii de pipe care preced stagiul A0
- [ ] d. Implementarea unui predictor de branch-uri cât mai bun
- [ ] e. micșorarea numărului de stagii de pipe care preced stagiul A0

Conform figurii de mai jos care este cea mai veche întrerupere din banda de asamblare ?



- [ ] a. Nici una deoarece nu există un sistem de prioritizare ca să ne dăm seama
- [ ] b. Overflow
- [ ] c. Illegal opcode
- [ ] d. Data address exception
- [ ] e. PC address exception

Se consideră implementarea unui procesor ca în figura de mai jos. De ce este MUX-ul din stagiul WB încercuit ?



- a. Avem nevoie de IR deoarece salturile și legăturile doar citesc și din acest motiv este nevoie de IR
- b. Destinația nu este codată în câmpul RD al instrucțiunii așa că IR este hardcodat cu 31
- c. Deoarece poate fi văzut ca un registru JAL către o destinație implicită
- d. Avem nevoie de IR deoarece salturile și legăturile doar scriu și din acest motiv este nevoie de IR

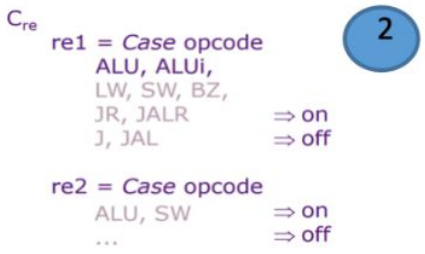
Se consideră programul de mai jos executat în bandă de asamblare. Când se poate face eliminarea instrucțiunilor 104 și 108 ?



- a. După faza de MEM
- b. După faza de IF
- c. După faza de WB
- d. După faza de ID
- e. După faza de EX

În cadrul determinării semnalului de control C_STALL etapa a doua presupune introducerea a două ecuații: re1 și re2. De ce ?

		source(s)	*destination*
ALU	rd ← (rs) func (rt)	rs, rt	rd
ALUI	rt ← (rs) op immediate	rs	rt
LW	rt ← M [(rs) + immediate]	rs	rt
SW	M [(rs) + immediate] ← (rt)	rs, rt	
BZ	*cond* (rs)		
	true: PC ← (PC) + immediate	rs	
	false: PC ← (PC) + 4	rs	
J	PC ← (PC) + immediate		
JAL	r31 ← (PC), PC ← (PC) + immediate		31
JR	PC ← (rs)	rs	
JALR	r31 ← (PC), PC ← (rs)	rs	31

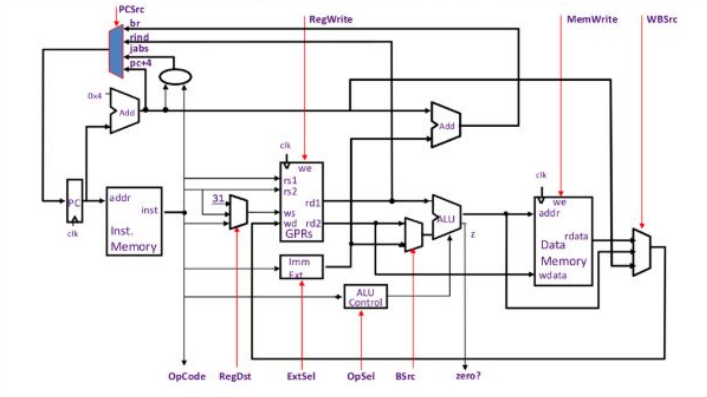


- a. Cele 2 ecuații reprezintă valorile realocate din cadrul unor registre folosite doar de instrucțiunile JALR și JAL
- b. Cele 2 ecuații reprezintă valorile realocate din cadrul unor registre folosite de anumite instrucțiuni
- c. Instrucțiunile JALR și JAL scriu întotdeauna și apoi nimic nu mai scrie fișierul de registre. ca atare re1 este folosit pentru citirea primului operand cât timp re2 este folosit pentru citirea celui de al doilea operand
- d. Avem nevoie obligatoriu doar de re1. re2 poate că se referă la operandul 2 dar cum se poate vedea și în figură el va trebui să acopere instrucțiunea SW care nu are un al doilea operand. Ar trebui eliminat pentru creșterea performanței benzii de asamblare

În cadrul figurii de mai jos, CP-ul este pus la finalul benzii de asamblare. De ce nu este el pus la finalul stagiului EX ?

- a. El se pune obligatoriu la finalul benzii de asamblare întodeauna
- b. Putem stabili dacă folosim sau nu CP exact ca în cazul procesoarelor OOO
- c. El trebuie pus cât mai devreme în banda de asamblare deoarece este bine ca excepțiile să stea cât mai puțin în banda de asamblare
- d. Ar fi mai bine să fie pus în faza de EX sau chiar în faza de ID
- e. El trebuie pus acolo unde nu mai pot fi luate excepții în pipe

Execuția unei instrucțiuni în cazul procesorului de mai jos fără bandă de asamblare:



- [ ] a. presupune necesitatea unui ciclu de ceas mic
- [ ] b. presupune necesitatea unui ciclu de ceas mare
- [ ] c. Figura nu prezintă semnale de control, deci nu se poate numi un procesor, ca atare nu poate executa instrucțiuni
- [ ] d. presupune trecerea prin toate elementele de la stânga la dreapta
- [ ] e. presupune trecerea prin toate elementele dar nu contează ordinea

Se presupune execuția următorului program și maparea instrucțiunilor în memoria cache de instrucțiuni conform figurii de mai jos. Totodată se presupune că nu avem JUMP. Așa cum se poate observa în figură, memoria poate conține 4 instrucțiuni pe o linie iar într-un ciclu de ceas se citesc 2 instrucțiuni odată (inclusiv pentru ciclul 4). Ce tip de memorie cache de instrucțiuni este necesară pentru a fi fezabil acest tip de citire ?

```
Cyc Addr Instr
0   0x000 OpA
0   0x004 OpB
1   0x008 OpC
1   0x00C J 0x100
…
2   0x100 OpD
2   0x104 J 0x204
…
3   0x204 OpE
3   0x208 J 0x30C
…
4   0x30C OpF
4   0x310 OpG
5   0x314 OpH
```

0x000	0	0	1	1
…				
0x100	2	2		
…				
0x200		3	3	
…				
0x300				4
0x310	4	5		

- a. O memorie cache de instrucțiuni single port cu o capacitate putere a lui 2
- b. O memorie cache de instrucțiuni single port
- c. O memorie cache de instrucțiuni dual port
- d. O memorie cache de instrucțiuni cu cel puțin 4 porturi
- e. O memorie cache de instrucțiuni cu cel mult 4 porturi